# Circular Sector Quantum Well

Complete Python source code — user enters the sector angle at runtime

## Dependencies: numpy, scipy, sympy, matplotlib

```
"""
Quantum particle in a circular-sector infinite potential well.

Region:
    0 < r < R, 0 < theta < alpha
Infinite potential outside the sector.

Dependencies:
    numpy, scipy, sympy, matplotlib

Install if needed:
    pip install numpy scipy sympy matplotlib
"""

import numpy as np
import sympy as sp
import matplotlib.pyplot as plt
import matplotlib.tri as mtri
from scipy.special import jv
from scipy.optimize import brentq

# --------------------------- USER SETTINGS ---------------------------

RADIUS = 1.0                # Sector radius R
NUM_STATES = 6              # Number of lowest states to display
NR = 100                    # Radial plotting resolution
NTHETA = 100                # Angular plotting resolution
HBAR = 1.0                  # Set to 1 for dimensionless units
MASS = 1.0                  # Particle mass

# ---------------------------------------------------------------------


def symbolic_solution():
    """Print the separated symbolic Schrödinger solution."""
    r, theta, alpha, R, mass, hbar = sp.symbols(
        "r theta alpha R M hbar", positive=True
    )
    ell, s = sp.symbols("ell s", positive=True, integer=True)
    nu = ell * sp.pi / alpha
    k = sp.symbols("k", positive=True)
    radial = sp.besselj(nu, k * r)
    angular = sp.sin(nu * theta)
    psi = sp.Function("psi")(r, theta)

    print("\nSYMBOLIC SCHRODINGER EQUATION (POLAR COORDINATES)")
    print("Inside the sector, V = 0:")
    print("-(hbar^2 / 2M) * [d²psi/dr² + (1/r)dpsi/dr")
    print("                    + (1/r²)d²psi/dtheta²] = E psi")
    print("\nBoundary conditions:")
    print("psi(r, 0) = psi(r, alpha) = psi(R, theta) = 0")
    print("\nSeparated angular eigenfunctions:")
    sp.pprint(sp.sin(ell * sp.pi * theta / alpha))
    print("\nBessel order:")
    sp.pprint(nu)
    print("\nRadial solution regular at r = 0:")
    sp.pprint(radial)
    print("\nCombined unnormalized eigenfunction:")
    sp.pprint(radial * angular)
    print("\nLet j_(nu,s) be the s-th positive zero of J_nu.")
    print("Then k = j_(nu,s)/R and the energy is:")
    energy = hbar**2 * sp.Symbol("j_nu_s", positive=True)**2 / (
        2 * mass * R**2
    )
    sp.pprint(energy)
    print("\nNormalized eigenfunction:")
    print("psi_(ell,s) = N J_nu(j_(nu,s) r/R) sin(nu theta)")
    print("N = 2 / (R sqrt(alpha) |J_(nu+1)(j_(nu,s))|)")
    print("where nu = ell*pi/alpha, ell = 1,2,... and s = 1,2,...")
    print("The Bessel zeros generally need numerical evaluation for")
    print("non-integer nu, so the program computes them numerically.\n")


def bessel_positive_zeros(nu, count):
    """Find the first `count` positive zeros of J_nu for nu > -1."""
    xmax = max(40.0, 3.0 * nu + 25.0, count * np.pi + nu + 10.0)
    # For nu > 0, the first positive zero lies beyond x = nu. Starting
    # near nu avoids mistaking floating-point underflow near x = 0 for a root.
    xmin = max(1e-5, nu)
    grid = np.linspace(xmin, xmax, 50000)
    vals = jv(nu, grid)
    roots = []

    for i in range(len(grid) - 1):
        if vals[i] == 0:
            root = grid[i]
        elif vals[i] * vals[i + 1] < 0:
            root = brentq(lambda z: jv(nu, z), grid[i], grid[i + 1])
        else:
            continue

        if root > 1e-6 and (not roots or abs(root - roots[-1]) > 1e-5):
```

```
            roots.append(root)
            if len(roots) >= count:
                return roots

    raise RuntimeError(
        f"Could not find {count} Bessel zeros for order {nu:.6g}. "
        "Increase the root-search range/resolution."
    )


def make_states(alpha, count):
    """Return the lowest states sorted by energy."""
    candidates = []
    ell_max = max(12, int(np.ceil(count * alpha / np.pi)) + 10)

    for ell in range(1, ell_max + 1):
        nu = ell * np.pi / alpha
        # A few extra radial roots ensure enough candidates for sorting.
        roots = bessel_positive_zeros(nu, count)
        for s, root in enumerate(roots, start=1):
            candidates.append({
                "ell": ell,
                "s": s,
                "nu": nu,
                "zero": root,
                "energy": HBAR**2 * root**2 / (2.0 * MASS * RADIUS**2),
            })

    candidates.sort(key=lambda state: state["energy"])
    return candidates[:count]


def normalization(state, alpha):
    """Positive normalization constant for the sector eigenfunction."""
    nu = state["nu"]
    root = state["zero"]
    denominator = abs(jv(nu + 1.0, root))
    return 2.0 / (RADIUS * np.sqrt(alpha) * denominator)


def wavefunction(state, r, theta, alpha):
    """Normalized real eigenfunction psi(r, theta)."""
    nu = state["nu"]
    root = state["zero"]
    norm = normalization(state, alpha)
    return norm * jv(nu, root * r / RADIUS) * np.sin(nu * theta)


def sector_mesh(nr, ntheta, alpha):
    """Triangular mesh covering a circular sector, including its centre."""
    radii = np.linspace(RADIUS / nr, RADIUS, nr)
    angles = np.linspace(0.0, alpha, ntheta + 1)

    points = [(0.0, 0.0)]
    ring_ids = []
    for radius in radii:
        ids = []
        for theta in angles:
            ids.append(len(points))
            points.append((radius * np.cos(theta), radius * np.sin(theta)))
        ring_ids.append(ids)

    triangles = []
    first_ring = ring_ids[0]
    for j in range(ntheta):
        triangles.append((0, first_ring[j], first_ring[j + 1]))

    for i in range(len(ring_ids) - 1):
        inner = ring_ids[i]
        outer = ring_ids[i + 1]
        for j in range(ntheta):
            triangles.append((inner[j], outer[j], outer[j + 1]))
            triangles.append((inner[j], outer[j + 1], inner[j + 1]))

    points = np.asarray(points)
    triangulation = mtri.Triangulation(
        points[:, 0], points[:, 1], np.asarray(triangles)
    )
    return triangulation, points


def plot_energy_levels(states, alpha):
    """Plot the lowest energy levels."""
    fig, ax = plt.subplots(figsize=(9, 5.5))
    for index, state in enumerate(states, start=1):
        energy = state["energy"]
        ax.hlines(energy, 0.1, 0.75, linewidth=2)
        ax.text(
            0.82, energy,
            f"State {index}: (ell,s)=({state['ell']},{state['s']})\n"
            f"E = {energy:.5g}",
            va="center", fontsize=9
        )

    ax.set_xlim(0, 2.7)
    ax.set_xticks([])
    ax.set_ylabel("Energy E")
    ax.set_title(
        f"Lowest {len(states)} energy levels "
        f"(sector angle = {np.degrees(alpha):.1f} degrees)"
```

```
    )
    ax.grid(axis="y", alpha=0.3)
    fig.tight_layout()


def plot_states(states, triangulation, points, alpha, quantity):
    """Plot wavefunctions or probability densities for all selected states."""
    cols = 3
    rows = int(np.ceil(len(states) / cols))
    fig, axes = plt.subplots(
        rows, cols, figsize=(4.5 * cols, 4.0 * rows), squeeze=False
    )

    r = np.hypot(points[:, 0], points[:, 1])
    theta = np.arctan2(points[:, 1], points[:, 0])
    # Keep theta in the sector's angular interval.
    theta = np.where(theta < 0, theta + 2.0 * np.pi, theta)

    for ax in axes.ravel():
        ax.axis("off")

    for index, (ax, state) in enumerate(zip(axes.ravel(), states), start=1):
        ax.axis("on")
        psi = wavefunction(state, r, theta, alpha)

        if quantity == "psi":
            vmax = max(np.max(np.abs(psi)), 1e-12)
            art = ax.tripcolor(
                triangulation, psi, shading="gouraud",
                cmap="RdBu_r", vmin=-vmax, vmax=vmax
            )
            title = "Wavefunction"
        else:
            density = psi**2
            art = ax.tripcolor(
                triangulation, density, shading="gouraud", cmap="viridis"
            )
            title = "Probability density"

        ax.set_title(
            f"State {index}: (ell,s)=({state['ell']},{state['s']})\n"
            f"E = {state['energy']:.5g}",
            fontsize=9
        )
        ax.set_aspect("equal")
        ax.set_xlabel("x")
        ax.set_ylabel("y")
        fig.colorbar(art, ax=ax, fraction=0.046, pad=0.04)

    for ax in axes.ravel()[len(states):]:
        ax.remove()

    fig.suptitle(
        f"{title} in a circular sector "
        f"(angle = {np.degrees(alpha):.1f} degrees)",
        fontsize=13
    )
    fig.tight_layout()


def main():
    try:
        angle_degrees = float(input('Enter sector angle in degrees (0 < angle <= 360): '))
    except ValueError:
        raise ValueError('Please enter a valid numeric angle.')

    if not (0.0 < angle_degrees <= 360.0):
        raise ValueError("ANGLE_DEGREES must be in the interval (0, 360].")
    if RADIUS <= 0 or MASS <= 0 or HBAR <= 0:
        raise ValueError("RADIUS, MASS and HBAR must all be positive.")
    if NUM_STATES < 1:
        raise ValueError("NUM_STATES must be at least 1.")

    alpha = np.radians(angle_degrees)
    symbolic_solution()
    states = make_states(alpha, NUM_STATES)

    print("LOWEST ENERGY EIGENSTATES")
    print("-" * 68)
    print("State   ell   s       Bessel zero       Energy")
    for i, state in enumerate(states, start=1):
        print(
            f"{i:5d}  {state['ell']:4d}  {state['s']:3d}"
            f"  {state['zero']:15.8f}  {state['energy']:12.7g}"
        )

    triangulation, points = sector_mesh(NR, NTHETA, alpha)
    plot_energy_levels(states, alpha)
    plot_states(states, triangulation, points, alpha, "psi")
    plot_states(states, triangulation, points, alpha, "density")
    plt.show()


if __name__ == "__main__":
    main()
```